

项目启动与研发协同小会

为什么我们需要规范

先统一

- 没有共同约定时，Git 冲突会越来越频繁。
- AI 生成的代码即使能跑，风格、边界和维护成本也可能失控。
- 前后端字段没先说清楚，容易陷入“我在等你改”的循环。
- 本地环境不一致时，不一定每一台机器都能跑
- 重复劳动尽量被工具吃掉。
- 问题在编码前或提交前暴露，防止联调时爆炸。
- 这样就可以早点下班。

总体共识

一些规则

- 格式化交给工具：保存时自动格式化，不在 PR 里讨论缩进、引号和换行。
- 基础代码交给 AI：样板代码、重复劳动、初稿结构，都优先让 AI 帮我们起步。
- 协作先对齐，再并行：字段、数据结构、职责边界先说清，再各自推进。
- 人类负责判断与验收：代码是否能解释、能维护、能交付，最终责任始终在提交者。

AI 算是副驾驶

能加速，但不能完全代替思考

AI 适合做…

- 起草重复代码、基础结构和测试样板。
- 快速总结已有文件和改动范围。
- 帮你把一个大任务拆成可执行的小步。
- 在你已有判断时，提供第二种写法或检查清单。

人需要亲自做…

- 防止盲从；提交前要知道代码到底在做什么。
- 必须能解释 AI 生成代码的逻辑、边界和影响面。
- 遇到 bug 时先自己理解报错，不要把同一段报错反复丢给 AI。
- 新栈和新 API 的问题，记得查官方文档，再让 AI 帮你理解和落地。

Prompt 习惯

先给上下文，再化整为零

错误示范

- 只说一句：帮我把这个页面写了。
- 不交代当前文件、技术栈、已有代码和约束。
- 出 bug 后，只把整段报错反复贴给 AI，希望它“猜中”。
- 一上来就让 AI 大改全项目，结果自己也看不懂。

更靠谱的写法

- 先说明项目上下文：在哪个文件、是什么栈、想解决什么问题。
- 再把任务化整为零：先分析，再拆步骤，再写某一小块。
- 提供输入、期望输出和已知限制，让 AI 不用靠猜。
- 要求 AI 先解释方案，再改代码；改完后总结风险点。

涉及新框架能力、陌生 API、官方约束时，文档优先，人和 AI 都要看文档。

后端规范

踩稳地基

环境与工具链

- 后端按 **Python 3.12+** 作为基本版本。
- 依赖管理统一使用 uv，和仓库已有说明保持一致。
- 本地环境、依赖安装、命令习惯尽量统一。
- 注意做代码格式化。

各种写法

- 写清类型标注，让函数签名和数据边界可读。
- 代码结构优先清楚、可替换，不要把逻辑糊成一团再靠注释补救。
- 注重模块化能力建设。

前端规范

界面逻辑与系统能力要分层

栈与职责

- 前端当前是 **React + Vite + Electron** 的桌面端形态。
- 渲染进程主要负责界面、状态和交互反馈。
- 主进程 / preload 负责更接近系统层的能力与桥接。

工程习惯

- 涉及文件、系统调用、原生能力时，统一走 **IPC**，不要让页面直接越界。
- 保存时做格式化，让 PR 更专注于逻辑本身。
- 页面改动尽量配合清楚描述，让 Review 的上下文成本更低。

前后端契约

Schema First

1. 先定字段：字段名、类型、可空性、枚举值先说清楚。
2. 再给样例：请求 / 响应最好带一个最小示例，减少理解偏差。
3. 然后并行开发：前端按契约调用和测试，后端按契约落实现，各写各的也能对得上。
4. 联调只验证实现：联调阶段重点看实现是否符合契约，而不是再回头争字段应该叫什么。

Git 与分支规范

保护主干，控制改动面

主干与分支

- 主干要受保护，**禁止直接 push** 到主分支。
- 一个分支只做一个主题，减少交叉污染。
- 分支命名尽量直观，例如：feat/yourname/setting-ui、fix/yourname/oserror-in-backend、chore/yourname/change-dependencies。
- 先拉最新代码，再开新分支，避免把历史问题一并带进来。

Commit 的底线

- AI 可以帮你起草 commit message，但人要最后把关。
- message 要说明“做了什么改动”。
- 让历史可读，未来的你和队友都能快速读懂。

PR 与 Review

小步快跑，比憋大招更友好

提 PR 的默认做法

- 如果能每天写一些代码，每天一 PR 是非常合适的频率，小 PR 比攒一坨更好。
- 小步快跑：让改动范围可讲清、可 review、可回退。
- PR 描述里最好写明背景、核心改动、验证方式。
- UI 相关改动最好补一句结果说明或截图线索。

Review 要看什么

- AI Review 是可行的，但一定要人工审核 AI 的意见。
- 格式问题交给工具，Review 重点看逻辑。
- 注意观察风险点：异常路径、回退成本、未来维护负担。

卡点求助

一个时间盒推荐

- 对于自己不熟悉的领域，AI 永远是提问的最好选择～
- 同一个问题卡住 1 小时左右，主动求助会是一个好选择。
- 求助不是能力差，而是避免时间浪费。
- 先做排查，再带着信息来问，效率最高。

推荐提问三要素

- 背景：你在做什么，目标是什么，相关文件或模块是什么。
- 已尝试：你已经查过什么、改过什么、为什么还不行。
- 现象：具体报错、当前结果、你预期应该发生什么。

总结

- 能交给工具的，不手动重复。
- 能交给 AI 的，不从零硬敲。
- 不能解释的代码，不要提交。
- 没对齐契约的协作，不要开始联调。